Solution: Extra Characters in a String

Problem Statement

Given a string `s` and an array of words `words`. Break string `s` into multiple non-overlapping substrings such that each substring should be part of the `words.` There are some characters left which are not part of any substring.

Return the minimum number of remaining characters in `s`, which are not part of any substring after string break-up.

Examples

1. **Example 1**:
    - **Input**: `s = "amazingracecar"`, `dictionary = ["race", "car"]`
    - **Expected Output**: `7`
    - **Justification**: The string `s` can be rearranged to form "racecar", leaving 'a', 'm', 'a', 'z', 'i', 'n', 'g' as extra.
2. **Example 2**:
    - **Input**: `s = "bookkeeperreading"`, `dictionary = ["keep", "read"]`
    - **Expected Output**: `9`
    - **Justification**: The words "keep" and "read" can be formed from `s`, but 'b', 'o', 'o', 'k', 'e', 'r', 'i', 'n', 'g' are extra.
3. **Example 3**:
    - **Input**: `s = "thedogbarksatnight"`, `dictionary = ["dog", "bark", "night"]`
    - **Expected Output**: `6`
    - **Justification**: The words "dog", "bark", and "night" can be formed, leaving 't', 'h', 'e', 's', 'a', 't' as extra characters.

Solution

The solution approach utilizes a dynamic programming strategy combined with a trie data structure. The dynamic programming aspect allows for efficiently keeping track of the minimum extra characters required at each position in the string `s`. This is achieved by building a bottom-up solution, where we start from the end of the string and work our way towards the beginning, calculating the minimum extra characters required for each substring.

The trie data structure is used to efficiently find and match the words from the `dictionary` within the string `s`. The combination of these two methods ensures that the solution is both time-efficient and space-efficient, as it avoids redundant computations and efficiently manages the storage of the dictionary words.

Step-by-Step Algorithm

1. **Initialize Trie**:
    - Construct a trie using the words from the `dictionary`. Each node in the trie represents a character, and a complete path from the root to a leaf node represents a word.
2. **Dynamic Programming Array**:
    - Initialize a dynamic programming (DP) array, `dp`, of length `n + 1`, where `n` is the length of the string `s`. This array will store the minimum number of extra characters required for the substring starting from each index.
3. **DP Calculation**:
    - Iterate backwards through the string `s`:
        - Set `dp[start]` to `dp[start + 1] + 1` initially. This represents the case where the current character is considered an extra character.
        - For each `start` position, iterate through the string to check if a word in the trie can be formed starting from this position.
        - If the current substring matches a word in the trie (`node.isEnd` is true), update `dp[start]` to be the minimum of its current value and `dp[end + 1]`, where `end` is the end of the matched word. This step ensures that we consider removing the matched word and count the rest as extra characters.
4. **Return Result**:
    - The value of `dp[0]` gives the minimum number of extra characters in the entire string `s`.

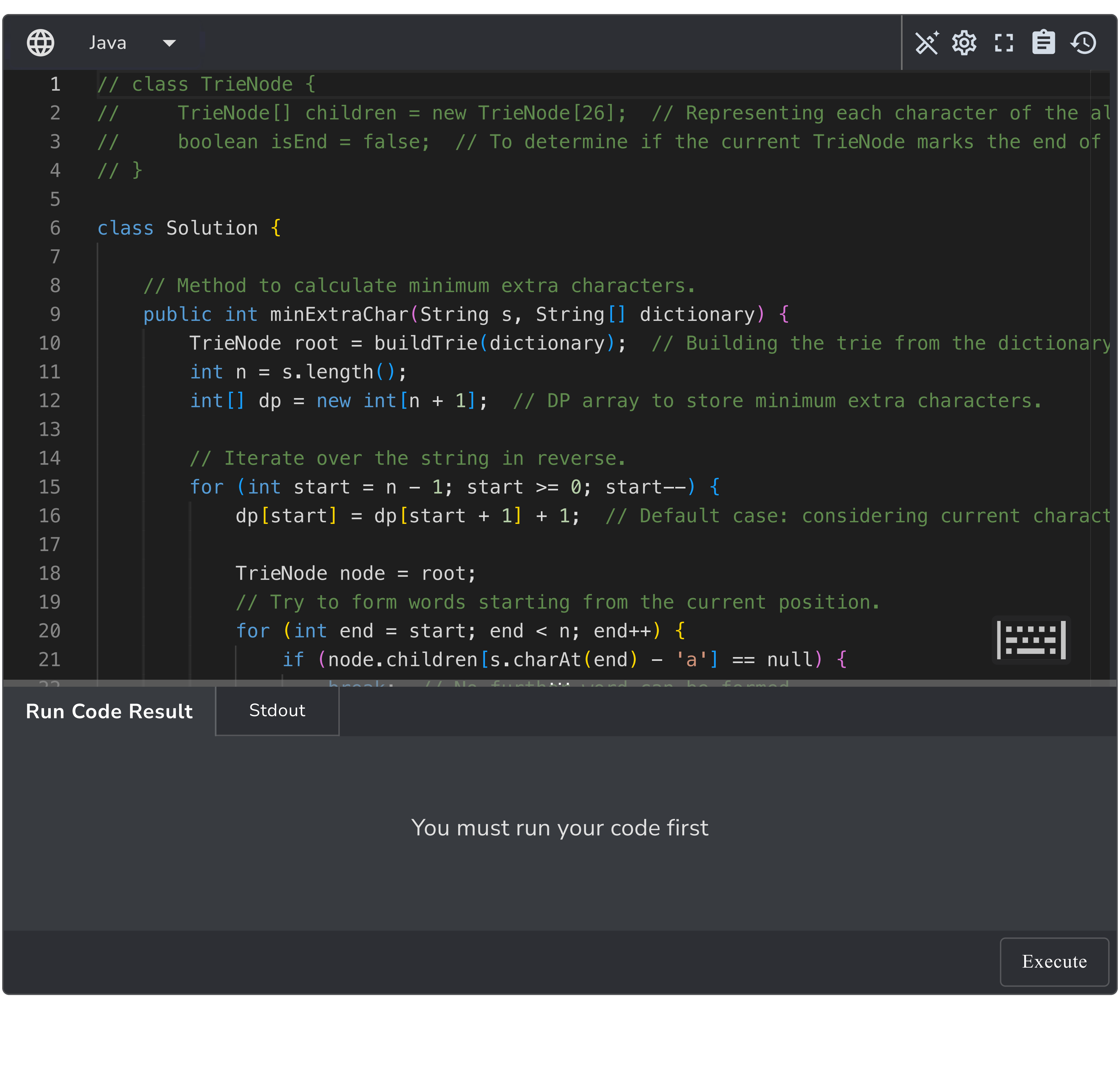
Algorithm Walkthrough

Let's consider the input s = `"bookkeeperreading"`, dictionary = `["keep", "read"]`

1. **Trie Construction**:
    - Build the trie with "keep" and "read".
2. **DP Initialization**:
    - Initialize `dp` array of length 18 (since "bookkeeperreading" has 17 characters).
3. **DP Calculation**:
    - Start from index 16 (last character 'g'):
        - For each index, try to form a word from the trie. For example, at index 12, the word "read" can be formed.
        - Update `dp[12]` to be the minimum of `dp[12]` and `dp[16]` (which is the end of "read" + 1).
4. **Iterate Backwards**:
    - Continue this process for each character in `s`. If a character does not form a word in the trie, `dp[start]` remains `dp[start + 1] + 1`.
5. **Final Result**:
    - `dp[0]` will have the minimum extra characters after processing the entire string.

This approach systematically checks each substring of `s` against the trie, and the dynamic programming array efficiently keeps track of the minimum extra characters required. The use of a trie ensures that each substring check is done in an optimized manner, avoiding unnecessary recomputations.

Code



```java
// class TrieNode {
//     TrieNode[] children = new TrieNode[26];  // Representing each character of the al
//     boolean isEnd = false;  // To determine if the current TrieNode marks the end of
// }

class Solution {

    // Method to calculate minimum extra characters.
    public int minExtraChar(String s, String[] dictionary) {
        TrieNode root = buildTrie(dictionary);  // Building the trie from the dictionary
        int n = s.length();
        int[] dp = new int[n + 1];  // DP array to store minimum extra characters.

        // Iterate over the string in reverse.
        for (int start = n - 1; start >= 0; start--) {
            dp[start] = dp[start + 1] + 1;  // Default case: considering current charact

            TrieNode node = root;
            // Try to form words starting from the current position.
            for (int end = start; end < n; end++) {
                if (node.children[s.charAt(end) - 'a'] == null) {
```

You must run your code first

Time and Space Complexity Analysis

Time Complexity

- **Trie Construction**: (O(W x L)), where (W) is the number of words in the dictionary and (L) is the average length of these words.
- **Dynamic Programming Calculation**: (O(n^2)), where (n) is the length of the input string `s`.
- **Total Time Complexity**: (O(W x L + n^2)).

Space Complexity

- **Trie Storage**: (O(W x L)), for storing the words in the trie.
- **Dynamic Programming Array**: (O(n)), for the array used in dynamic programming.
- **Total Space Complexity**: (O(W x L + n)).

← Previous
Extra Characters In A String (Medium)

Next →
Search Suggestions System (Medium)

☐ Mark as Completed